

회의록이 아니라 정산서입니다 — 1인 셀러가 매달 잃던 저녁을 되찾은 법

② 활용사례 분야 · 참가 구분: 개인 · 사용 도구: 생성형 AI(Claude) · 적용 영역: 가게 운영 업무

1. 활용 배경

오픈마켓에서 혼자 물건을 팝니다. 매달 정산일이 오면 저녁이 통째로 사라졌습니다. 마켓이 주는 정산 파일에는 **얼마가 입금되는지만** 적혀 있고 원가도 광고비도 거기 없기 때문입니다. 엑셀을 열고 상품명을 맞추고 원가를 붙이고 수수료를 빼고 광고비를 나눠 넣었습니다. **한 달치에 서너 시간**이 걸렸고, 무엇보다 **그 숫자가 맞는지 확인할 방법이 없었습니다.**

저는 개발자가 아니라 코드를 못 씁니다. 그래서 생성형 AI에게 정산 파일을 읽는 도구를 같이 만들자고 했고, 나중에는 블로그 글쓰기까지 넓혔습니다.

2. 활용 방법 및 프롬프트 노하우

제 프롬프트는 **맥락** → **지시** → **검증 장치** 세 뼈대를 가집니다. 마지막 것이 핵심이고, 그것 때문에 결과를 믿을 수 있게 됐습니다.

① 맥락을 먼저 대고 지시한다

"오픈마켓 정산 엑셀을 넣으면 상품별로 얼마가 남는지 나오는 도구를 만들려고 합니다. 문제는 마켓마다 컬럼 이름이 다르다는 겁니다. '정산금액', '결제금액', '실판매가'가 전부 다른 뜻일 수 있습니다. 컬럼 이름을 자동으로 알아보는 사전부터 만들어 주세요. 제가 확인할 수 있게 어떤 이름을 무엇으로 봤는지 화면에 표시해 주세요."

코드는 모르지만 **정산 파일이 어떻게 생겼는지는 압니다.** 그 앞을 앞에 대는 것이 지시보다 먼저였습니다.

② 답을 아는 예제를 먼저 만들게 한다 — 환각 검증 장치

"이 계산이 맞는지 제가 확인할 방법이 없습니다. 순이익이 얼마여야 하는지 **제가 미리 아는** 가짜 정산 파일을 만들어 주세요. 상품별 정답도 같이 적어 주세요. 그걸로 도구 결과와 대조하겠습니다."

제가 찾은 가장 중요한 방법입니다. AI에게 계산을 맡기면 결과가 그럴듯하게 나오는데 **틀렸는지 알 수가 없습니다.** 그래서 코드를 잘 받는 것보다 **틀린 걸 잡아낼 자를 먼저 만드는 쪽**에 프롬프트를 썼습니다. 실제로 이 대조에서 오류 두 가지가 잡혔습니다. 수수료 부호가 마켓마다 달라 한쪽에서는 수수료가 이익으로 더해지고 있었고, 배송비는 주문 단위인데 상품 행마다 반복돼 그냥 더하면 부풀려졌습니다.

③ 숫자는 출처를 열어 보게 한다

"이 수수료율을 어디서 봤습니까. 그 페이지를 직접 열어서 확인해 주세요. 못 열면 못 열었다고 하세요. 추측한 값은 쓰지 마세요."

블로그에 수수료율을 적었다가 틀린 적이 있습니다. 공식 표를 열어 보니 제가 적은 범위가 실제 최고치보다 높았습니다. 그때부터 **출처를 직접 열지 않은 숫자는 쓰지 않는다**를 규칙으로 걸었습니다. 가장 위험한 건 코드가 아니라 **그럴듯한 숫자**입니다. 코드는 틀리면 에러가 나지만, 숫자는 틀려도 아무 일도 일어나지 않습니다.

3. 적용 전후 비교

	AI 도입 전	AI 도입 후
정산 한 달치 정리	서너 시간	몇 분

	AI 도입 전	AI 도입 후
결과를 믿을 수 있나	확인할 방법 없음	정답 아는 예제와 대조
적자 상품을 아는가	모름	자동으로 먼저 표시
블로그 글 한 편	두세 시간	한 시간 안쪽

숫자보다 크게 달라진 것이 있습니다. 제가 다루는 상품은 서른 개가 안 돼서 다 안다고 생각했는데 아니었습니다. **매출 상위에 있으면서 실제로는 손해인 상품이 있다는 걸 알게 됐습니다.** 잘 팔리는 상품이 번 돈에 묻혀 총액에서는 보이지 않습니다. 계산이 몇 분으로 줄자 비로소 보였습니다.

그리고 **일의 종류가 바뀌었습니다.** 전에는 표와 문장을 만드는 데 시간을 썼고, 지금은 무엇을 접고 무엇을 밀지 정하는 데 씁니다. 두 번째가 원래 해야 할 일이었습니다.

4. 노하우 공유 (재현 가능한 워크플로우)

1. 내 데이터의 함정부터 말로 적는다. 어느 컬럼이 헛갈리는지, 어디서 숫자가 들어지는지. 이걸 AI가 모르고 나만 압니다.
2. 답을 아는 예제부터 만들게 한다. 도구보다 자를 먼저 만듭니다.
3. 도구를 만들고 예제로 대조한다. 한 줄이라도 어긋나면 원인을 찾을 때까지 넘어가지 않습니다.
4. 내 실제 파일로 한 번 더 돌린다. 예제에 없던 함정이 여기서 나옵니다.
5. AI가 말한 숫자는 출처를 열어 확인한다. 못 열면 쓰지 않습니다.

주의할 점 하나. **“자연스럽게 써 줘”처럼 느낌으로 부탁하면 말투만 바뀌고 구조는 그대로입니다.** 다 쓰게 한 뒤 손으로 고쳤더니 고치는 자리가 매번 같았고, 목록으로 만드니 10분이면 끝났습니다. **프롬프트를 다듬는 것보다 고칠 자리를 목록으로 만드는 쪽이 빨랐습니다.**

5. 확산 가능성 및 소감

정산은 예일 뿐입니다. 매달 같은 양식의 파일을 받아 손으로 정리하는 일 — 소상공인의 매입·매출 정리, 관리비 내역 등 — 에 순서 그대로 옮겨집니다. 개발자가 아닌데도 된 이유는 하나입니다. **코드는 몰라도 내 데이터가 어떻게 생겼는지는 내가 제일 잘 알기 때문입니다.** AI는 그 앎을 형태로 옮겨 주는 쪽이었지 대신 알아 주는 쪽이 아니었습니다.

개선 제안 — 정부·공공 AI 교육에 ‘검증 실습’을 넣어 주십시오. 지금의 AI 교육은 대부분 “이렇게 물어보면 이런 답이 나옵니다”에서 끝납니다. 그러나 위험한 지점은 답을 못 받는 것이 아니라 **틀린 답을 그럴듯하게 받는 것입니다.** 한 시간짜리 교육이라면 마지막 15분은 “정답을 아는 예제로 AI의 답을 대조해 보기”에 써야 합니다. 전문 지식 없이 누구나 할 수 있고, 저에게 효과가 가장 컸던 단계입니다.

6. 출처 표기 및 저작권 준수

본 사례는 생성형 AI(Claude, 2026년 이용 기준)를 활용하였습니다. 도구의 코드와 글의 초안을 AI로 생성하였고, **모든 계산 결과는 정답을 아는 예제 파일과 대조하여 본인이 직접 검증한 뒤** 사용하였습니다. 판단과 결론은 본인의 것입니다. 거래 자료는 AI에 넘기지 않았습니다. 정산 파일에 주문번호와 매출이 들어 있어, 계산 전부를 브라우저 안에서만 하도록 만들었습니다. **파일이 기기 밖으로 나가지 않습니다.** 첨부 화면은 전부 예시 데이터입니다.